title: “EGN475 W1A1 — Flask Prototyping Environment” author: “Kouis Younes”

date: “2026-11-01”

Submission Draft and Evidence Report

Course activity: EGN475 W1A1 — Prototyping Environment
Project: Café Kouis Flask Prototype
Student: Kouis Younes
Prepared with disclosed automated assistance

> **Academic-integrity note.** This is an evidence-based preparation draft, not a substitute for the student’s own understanding or course-policy obligations. The project source, automated tests, and screenshots were created and verified in a Manus Ubuntu sandbox. The student should read the code, run it in the required course environment, and rewrite the personal answers below in their own words before submission.

1. Completion status

Requirement	Status	Evidence / note
Flask virtual environment	Completed in sandbox	`.venv/` created with Python 3.12.3; Flask 3.1.3 installed
`.gitignore` for `.venv/` and `__pycache__/`	Completed	Both required entries are present and were verified with `git check-ignore`
Working `app.py` with `/`	Completed	Flask test client and real browser returned HTTP 200
Working `/about` route	Completed	Flask test client and real browser returned HTTP 200
Three meaningful Git commits	Completed locally	Three core development commits, plus a separate evidence/report commit, are described in Section 4
Actual testing screenshots	Completed	Browser screenshots in Section 3
VS Code screenshot	Not available	Windows desktop / VS Code was not accessible from this sandbox; none fabricated
Copilot interaction	Not available	Copilot Chat was not accessible; no interaction is claimed
Public GitHub repository	Pending	GitHub connector authorization was requested but not approved; no repository was created or pushed

2. Prototype implementation

The application follows a small and readable Flask structure:

File / folder	Responsibility
`app.py`	Creates the Flask app and maps URLs to route functions
`templates/index.html`	Home-page HTML returned for `/`
`templates/about.html`	About-page HTML returned for `/about`
`static/style.css`	Shared visual styling for both pages
`tests/test_app.py`	Automated tests for the success, navigation, and 404 cases
`requirements.txt`	Declares `Flask ⩾ 3.0,<4.0`

The route implementation is intentionally minimal:

```
@app.get("/")
def home():
    return render_template("index.html", page_title="Café Kouis | Home")

@app.get("/about")
def about():
    return render_template("about.html", page_title="Café Kouis | About")
```

When a visitor requests `/` or `/about`, Flask selects the decorated function, calls `render_template`, reads the specified file from the `templates` folder, and returns the rendered HTML response. The templates use `url_for` to create the navigation and stylesheet URLs, which avoids hard-coding endpoint locations throughout the HTML.

3. Testing evidence

Automated tests

The automated test suite used Python's standard `unittest` framework. It tested four behaviors:

1. `/` returns HTTP 200 and expected home-page content.
2. `/about` returns HTTP 200 and expected about-page content.
3. The home page contains a link to `/about`.

4. A nonexistent route returns HTTP 404.

Actual result: 4 tests ran and all passed (`OK`) in 0.007 seconds. The recorded environment was Python 3.12.3 with Flask 3.1.3.

Browser verification

The application was started locally on `127.0.0.1:5050` in the sandbox. `curl` received HTTP 200 for both required routes, and Chromium headless browser captured the following actual rendered pages.

Figure 1. Actual browser capture of the `/` home page.

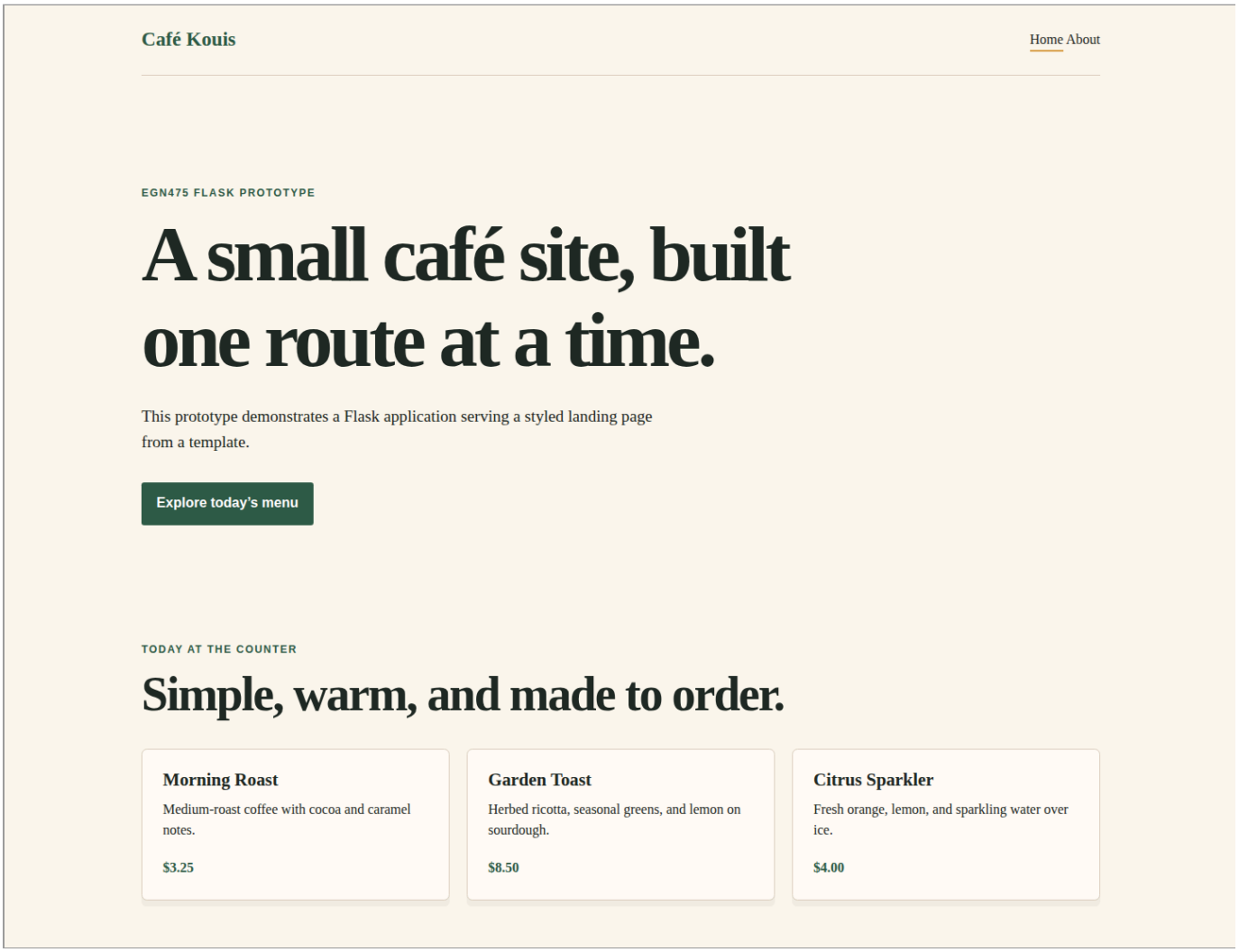
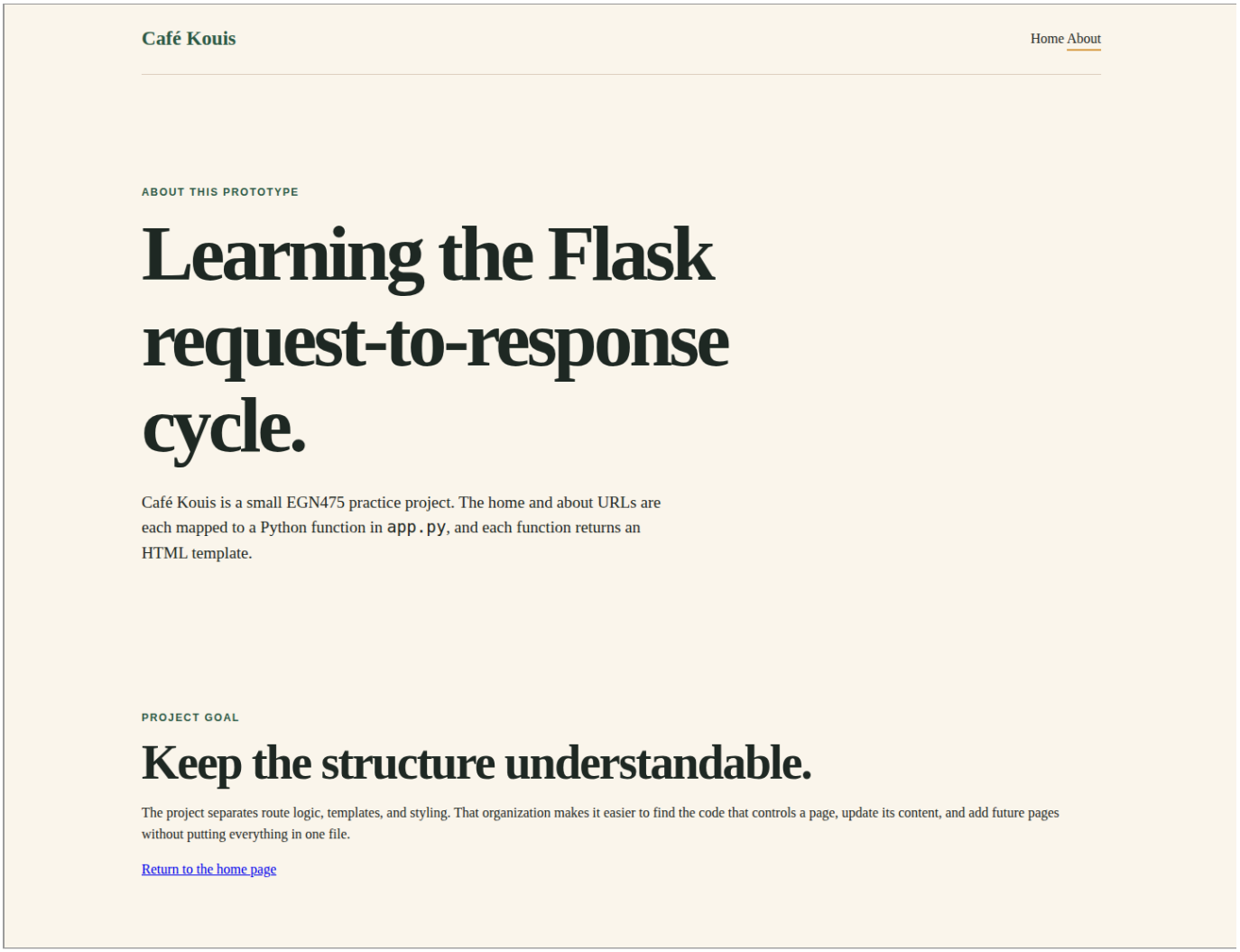


Figure 2. Actual browser capture of the `/about` page.



4. Git evidence

Three core development commits were created in chronological development order:

Commit	Message	Meaningful change
`fe9b484`	`Initialize Flask cafe home page`	Added the virtual-environment ignore rules, Flask application, home template, stylesheet, and requirements file.
`6df6558`	`Add about page route and navigation`	Added `/about`, its template, and navigation between the two pages.
`8021f83`	`Add route tests and project documentation`	Added the automated test suite, Windows run instructions, Q&A drafts, and the truthful assistance log.

After browser verification, a fourth documentation commit packages the actual screenshots and this report. The three core commits above satisfy the requested development-history requirement, while the fourth preserves the final evidence with the project. The working tree was verified clean immediately after the third core commit. The `.venv/` and `__pycache__/` paths were not tracked. To avoid misrepresenting authorship, the commits use the visible local identity `Manus Coursework Assistant <dev-agent@manus.ai>`, not the student's identity.

5. Personal-explanation drafts

> **Revise before submitting.** The instructor's exact four questions were not supplied. These are draft responses for common questions about this prototype, not a representation of the original assignment wording.

Draft 1 — Why use a virtual environment?

A virtual environment keeps this project's Python packages separate from packages installed for other projects. In this project, Flask is installed inside the `.venv` folder, so the application can use the version listed in `requirements.txt` without changing the system-wide Python installation. This makes the setup more repeatable because another developer can create a new `.venv` and install the same project requirements.

Draft 2 — What does `.gitignore` do?

The `.gitignore` file tells Git not to track files that are generated locally or that do not belong in the source repository. Here, it excludes `.venv/`, where the local virtual environment stores installed packages, and `__pycache__/`, where Python stores compiled cache files. Ignoring those folders keeps commits focused on source code and avoids adding machine-specific or automatically generated files.

Draft 3 — How do the Flask routes work?

Flask uses decorators above Python functions to associate each URL with a route. The `@app.get("/")` decorator maps the home URL to the `home()` function, while `@app.get("/about")` maps the about URL to `about()`. Each function calls `render_template`, which loads the matching HTML file from the `templates` folder and sends it back to the browser as the response.

Draft 4 — Why make separate Git commits?

Separate commits create a readable development history. The first commit establishes the basic Flask setup and home page, the second adds the about-page feature, and the third adds tests

and documentation. This makes it easier to review what changed at each stage, return to an earlier version if a problem occurs, and show that the prototype developed in meaningful steps instead of appearing as one unexplained final upload.

6. Truthful AI interaction log

Item	What actually occurred
Initial request	The user asked Manus to build the Flask prototype, prepare course materials, and publish a public GitHub repository if authorized.
Environment	Work occurred in a Manus Ubuntu sandbox, which reported Python 3.12.3 and Git 2.43.0. The stated Windows 11 / Python 3.12.10 / Git 2.56.0 environment was not directly accessed.
Code assistance	Manus created the starter Flask code, templates, CSS, tests, README, Q&A preparation drafts, and this report source.
Testing	Manus ran the Flask test client, executed the four automated tests, started the local Flask server, checked both routes with HTTP requests, and captured both pages using Chromium headless.
Git	Manus created three local commits, using a transparent automated-assistance author identity.
GitHub	Manus requested the available GitHub connector. Authorization was not accepted, so no GitHub repository, remote URL, or push was created.
VS Code and Copilot	Neither tool was available from the sandbox. No VS Code or Copilot screenshots, interactions, or results are included or claimed.
Provided source materials	No assignment instruction file or pre-existing screenshot was available in the sandbox.

7. Student actions still required

1. Review every source file and run the application and tests in the environment required by the instructor.

2. Capture authentic Windows/VS Code and, if required by the instructor, Copilot evidence while completing or reviewing the project yourself.
3. Replace the four drafts with original answers that accurately describe your own understanding and actions.
4. Follow the course's AI-use and citation policy. Retain, edit, or submit the disclosure only as that policy requires.
5. Authorize GitHub access or create `egn475-cafe-kouis` manually as a **public** repository; then add the remote and push the verified local `main` branch. Verify the repository URL and public visibility afterward.

Appendix: Local commands used for verification

```
.venv/bin/python -m unittest discover -s tests -v
curl http://127.0.0.1:5050/
curl http://127.0.0.1:5050/about
git log --oneline --decorate
git check-ignore -v .venv/bin/python __pycache__/app.cpython-312.pyc
```

The full automated-test console output is available in `evidence/automated-test-output.txt` in the working project.